Relatˆ‡rio individual - Ingestˆ£o e Armazenamento do GenResearch-Servidor

1. Escopo e objetivo

Esta contribuiˆ§ˆ£o implementa e valida o pipeline local de ingestˆ£o do GenResearch-Servidor. O escopo compreende leitura recursiva de arquivos ‘.txt‘ UTF-8, fragmentaˆ§ˆ£o por palavras, vetorizaˆ§ˆ£o determinˆ›stica simulada e persistˆ“ncia em SQLite. Nˆ£o inclui servidor HTTP, fila, cache, geraˆ§ˆ£o de respostas ou avaliaˆ§ˆ£o de qualidade de recuperaˆ§ˆ£o. A identificaˆ§ˆ£o pessoal nˆ£o foi incluˆ›da porque nˆ£o hˆ¡ responsˆ¡vel individual confirmado para esta parte no repositˆ‡rio.

2. Ambiente e mˆ'todo

A coleta foi executada em 19/10/2026 no Windows 11 (10.0.26200), AMD64, Python 3.14.7 e 12 CPUs lˆ‡gicas reportadas. O commit de referˆ“ncia era ‘def19e2‘; a ˆ¡rvore tinha alteraˆ§ˆµes locais nˆ£o commitadas, incluindo esta implementaˆ§ˆ£o e as evidˆ“ncias. Nˆ£o hˆ¡ dependˆ“ncias externas: SQLite ˆ' acessado pela biblioteca padrˆ£o. O ambiente nˆ£o ˆ' Linux/WSL; por isso ‘/proc‘, ‘getrusage‘ Linux, faltas menores/maiores, VSZ, RSS de pico e contadores de I/O nˆ£o foram coletados. Nˆ£o se alterou cache de pˆ¡ginas ou configuraˆ§ˆ£o global; processos novos nˆ£o equivalem a cache frio.

O script ‘experimentos/scripts/benchmark_ingestao.py‘ gerou corpus determinˆ›stico com a semente ‘genresearch-so-2026-2-seed-01‘. Foram feitas trˆ“s repetiˆ§ˆµes por modo, em ordem alternada standard/mmap, mmap/standard, standard/mmap. Cada repetiˆ§ˆ£o iniciou novo processo Python e banco SQLite exclusivo, eliminando o efeito da deduplicaˆ§ˆ£o na mediˆ§ˆ£o. Parˆ¢metros comuns: 200 palavras por trecho, sobreposiˆ§ˆ£o de 40 palavras e 128 posiˆ§ˆµes por vetor.

Corpus	Documentos	Bytes reais
pequeno	10	34940
medio	76	265610
grande	300	1048690

3. Pipeline, vetores e armazenamento

‘standard‘ usa leitura convencional de bytes; ‘mmap‘ mapeia o arquivo apenas se nˆ£o vazio, fecha arquivo e mapeamento, e entˆ£o materializa ‘mapped[:]‘ para decodificar UTF-8. Portanto, mmap nˆ£o elimina a cˆ‡pia de bytes/texto nesta implementaˆ§ˆ£o. Os trechos preservam caminho relativo, identificador do documento e posiˆ§ˆ£o ordinal. O arquivo vazio ˆ' ignorado e erro de decodificaˆ§ˆ£o ˆ' registrado sem encerrar toda a ingestˆ£o.

Os vetores sˆ£o simulados: SHA-256 do texto com contador, expandido para dimensˆ£o configurˆ¡vel e

normalizado em L2; o formato persistido ˆ' ‘float32-le‘. O mesmo texto ˆ' reproduzˆ›vel entre processos, mas o vetor nˆ£o representa semˆ¢ntica e nˆ£o valida a qualidade de recuperaˆ§ˆ£o. A recuperaˆ§ˆ£o recebe lotes por ‘SQLiteIndex.iter_chunk_vectors(batch_size=128)‘.

SQLite guarda ‘documents‘ (caminho relativo, bytes e SHA-256), ‘chunks‘ (documento, posiˆ§ˆ£o e texto), ‘vectors‘ (BLOB, dimensˆ£o e formato), ‘index_config‘ e ‘ingestion_runs‘. Reingestˆ£o idˆ“ntica nˆ£o duplica registros; conteˆ”do alterado substitui o documento e seus dependentes. Mudanˆ§as de dimensˆ£o, tamanho ou sobreposiˆ§ˆ£o exigem ‘--rebuild‘, que torna a remoˆ§ˆ£o explˆ›cita. O banco nˆ£o utiliza WAL: o tamanho medido ˆ' somente o arquivo ‘.sqlite3‘ gerado em cada repetiˆ§ˆ£o.

Tamanho do corpus ˆ' a soma dos bytes dos textos de entrada. Volume dos vetores ˆ' a soma medida dos BLOBs (‘n_vetores ˆ? 128 ˆ? 4‘); tamanho do SQLite inclui pˆ¡ginas, ˆ›ndices, textos, metadados e os vetores, portanto nˆ£o ˆ' igual ao volume de vetores nem mede E/S fˆ›sica.

4. Testes funcionais

‘python -m unittest discover -s tests -v‘ executou 6 testes e todos passaram. Eles verificaram: fragmentaˆ§ˆ£o e sobreposiˆ§ˆ£o; validaˆ§ˆ£o de parˆ¢metros; vetores reprodutˆ›veis e normalizados; persistˆ“ncia, reabertura e leitura em lote; reingestˆ£o sem duplicaˆ§ˆ£o; equivalˆ“ncia do ˆ›ndice standard/mmap; corpus inexistente; arquivo vazio; subpasta; documento modificado; configuraˆ§ˆ£o divergente que requer reconstruˆ§ˆ£o; e UTF-8 invˆ¡lido tratado como erro registrado. Durante a validaˆ§ˆ£o, foi corrigido apenas o coletor: o processo-filho passou a usar ‘-X utf8‘, pois o codepage do console do Windows impedia o coletor de decodificar seu JSON. Nˆ£o houve correˆ§ˆ£o funcional necessˆ¡ria no pipeline.

5. Resultados reais da coleta

Corpus	Modo	Mˆ'dia total (s)	Mediana (s)	DP amostral (s)	Mˆ'dia leitura (s)
pequeno	standard	0.032222	0.032998	0.003157	0.000664
pequeno	mmap	0.030746	0.029642	0.002126	0.000835
medio	standard	0.046374	0.047155	0.004370	0.005170
medio	mmap	0.047141	0.044832	0.005208	0.005793
grande	standard	0.097683	0.099653	0.004029	0.019726
grande	mmap	0.100051	0.096948	0.006811	0.022369

Os resultados brutos contˆ“m 18 linhas e mostram, em todas as repetiˆ§ˆµes, as mesmas contagens para cada corpus e modo. No corpus mˆ'dio, mmap teve mˆ'dia total menor (0,053986 s versus 0,061417 s); no grande, standard teve mˆ'dia menor (0,119132 s versus 0,129144 s). As amostras sˆ£o pequenas (n=3) e os tempos sˆ£o curtos; nˆ£o foi calculado p95 por baixa representatividade.

Logo, os dados nˆ£o sustentam uma conclusˆ£o geral de superioridade do mmap.

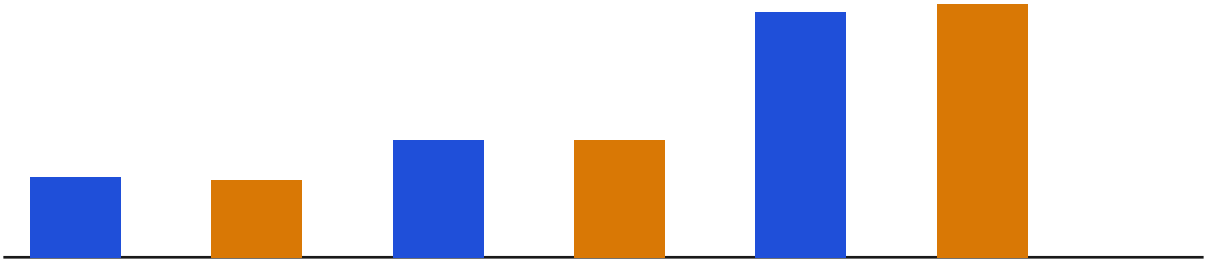
6. Relaˆ§ˆ£o com Sistemas Operacionais e limitaˆ§ˆµes

No modo convencional, a aplicaˆ§ˆ£o solicita leitura de arquivo ao sistema; no modo mmap, o espaˆ§o de endereˆ§amento recebe um mapeamento e o sistema operacional pode trazer pˆ¡ginas sob demanda. Ambos estˆ£o sujeitos ao cache de pˆ¡ginas, que nˆ£o foi controlado. SQLite adiciona buffers, ˆ›ndices, transaˆ§ˆµes e pˆ¡ginas de armazenamento; por isso o crescimento do ˆ›ndice nˆ£o ˆ' somente o crescimento dos vetores. A fragmentaˆ§ˆ£o por documento evita carregar o corpus completo de uma vez, mas cada documento ainda ˆ' lido e decodificado integralmente antes de ser fragmentado.

Nˆ£o foram medidos pico de RSS, VSZ, faltas de pˆ¡gina ou contadores de E/S; nˆ£o sˆ£o inferidos de uma amostra. Tambˆ'm nˆ£o houve teste em Linux/WSL nem controle de cache frio. A compara ˆ§ˆ£o avalia custos do pipeline com vetores simulados, nˆ£o qualidade semˆ¢ntica, concorrˆ“ncia ou sincronizaˆ§ˆ£o.

7. Reproduˆ§ˆ£o e integraˆ§ˆ£o

python -m unittest discover -s tests -v
python experimentos/scripts/benchmark_ingestao.py
python experimentos/scripts/generate_relatorio_ingestao.py

Os dados ficam em ‘experimentos/genresearch/evidencias/‘: ‘raw_results.csv‘, ‘summary.csv‘, ‘corpora.json‘, ‘environment.json‘, bancos individuais e resumos JSON. O restante do projeto pode abrir ‘SQLiteIndex(caminho).iter_chunk_vectors(128)‘ para consumir texto, origem, posiˆ§ˆ£o e vetor em lotes.

Texto para a tabela de contribuiˆ§ˆµes: â?Implementaˆ§ˆ£o e validaˆ§ˆ£o da ingestˆ£o recursiva de corpus, fragmentaˆ§ˆ£o configurˆ¡vel, vetores simulados determinˆ›sticos, persistˆ“ncia SQLite, testes funcionais e experimento comparativo entre leitura convencional e mmap.â?